

Cross_Validation

Math 742

2026-02-09

Training and Test Errors

Definition: Training error rate is the error rate calculated by building a model from the observations available and then predicting the response values and comparing them to the actual response values and averaging the errors.

Definition: *Test error rate* is the average error for all data values in the test set using the model that was fit using the training data.

→ Why don't we just use training error?

At first glance, it might seem natural to select the model with the smallest training error. However, this approach can be misleading, especially for flexible models.

To illustrate this, consider the following example

Let $Y = \sin(x) + \varepsilon$. Make some random data and split the data into 2 sets, 80% for training and 20%. Fit 3 models:

- Linear model
- Degree 3 polynomial
- Degree 12 polynomial

Computing training and test errors for each of these.

```
# -----  
# Overfitting example: training vs test error  
# True model:  $y = \sin(\pi x) + \text{epsilon}$   
# Models: linear, degree 3 polynomial, degree 12 polynomial  
# -----  
  
set.seed(9874)  
  
# 1. Generate data  
n <- 50  
x <- runif(n, min = -1, max = 1)  
eps <- rnorm(n, mean = 0, sd = 1.0)  
y <- sin(pi * x) + eps  
  
dat <- data.frame(x = x, y = y)  
  
# 2. Train/test split: 80% / 20%  
train_idx <- sample(seq_len(n), size = round(0.8 * n), replace = FALSE)  
train <- dat[train_idx, ]  
test  <- dat[-train_idx, ]  
  
# 3. Fit models
```

```

fit_lm <- lm(y ~ x, data = train)
fit_p3 <- lm(y ~ poly(x, 3, raw = TRUE), data = train)
fit_p12 <- lm(y ~ poly(x, 12, raw = TRUE), data = train)

# 4. Prediction function
mse <- function(actual, predicted) {
  mean((actual - predicted)^2)
}

# 5. Predictions
train_pred_lm <- predict(fit_lm, newdata = train)
test_pred_lm <- predict(fit_lm, newdata = test)

train_pred_p3 <- predict(fit_p3, newdata = train)
test_pred_p3 <- predict(fit_p3, newdata = test)

train_pred_p12 <- predict(fit_p12, newdata = train)
test_pred_p12 <- predict(fit_p12, newdata = test)

# 6. Results table
results <- data.frame(
  Model = c("Linear model", "Degree 3 polynomial", "Degree 12 polynomial"),
  Training_MSE = c(
    mse(train$y, train_pred_lm),
    mse(train$y, train_pred_p3),
    mse(train$y, train_pred_p12)
  ),
  Test_MSE = c(
    mse(test$y, test_pred_lm),
    mse(test$y, test_pred_p3),
    mse(test$y, test_pred_p12)
  )
)

results$Training_MSE <- round(results$Training_MSE, 4)
results$Test_MSE <- round(results$Test_MSE, 4)

print(results)

```

```

##           Model Training_MSE Test_MSE
## 1   Linear model      0.9445   0.6525
## 2 Degree 3 polynomial    0.8374   0.6570
## 3 Degree 12 polynomial    0.5408 1322.6507

```

The extremely large test error for the degree 12 model occurs because highly flexible models can produce very unstable predictions outside the training data, leading to large errors.

Notice that training error is minimized by construction, so it will always favor more complex models. AND, we can clearly see that the most flexible model generalizes poorly.

$$\mathbb{E}[\text{Training Error}] < \mathbb{E}[\text{Test Error}]$$

This occurs because the same data is used to both fit the model and evaluate its performance, leading to an overly optimistic estimate.

Cross validation is a method of estimating the *test error* by holding out a portion of the training data to use for testing.

What do we do?

- Best Solution: a large designated test set. Hard to find...
- Use methods that penalize the training error rate, either by accounting for the number of parameters in the model or penalizing you if your variability is too high (e.g. AIC, BIC).
- Validation - we hold out some training data in order to use as a test set.

Drawbacks to the validation set approach

- The validation estimate of the test error can be highly variable. This happens because you randomly select values to hold out for the test set.
- Your model fit isn't as good because you're not using all of the data. Some of the data is used for the test set.
- Because you are haven't fit your model on all of the data, the following can happen:

→ You have high variance depending on the split.

→ The validation error can be biased slightly upward because the model is trained on less data.

Car Example

To illustrate cross validation, I will use the cars data set.

```
library(readr)
cars <- read_csv("Data/cars.csv")
head(cars)

## # A tibble: 6 x 19
##   Car      Sports  SUV Wagon Minivan Pickup  AWD  RWD Retail Dealer Engine
##   <chr>      <dbl> <dbl> <dbl>  <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Acura 3.5 ~      0      0      0      0      0      0      0 43755 39014 3.5
## 2 Acura 3.5 ~      0      0      0      0      0      0      0 46100 41100 3.5
## 3 Acura MDX      0      1      0      0      0      1      0 36945 33337 3.5
## 4 Acura NSX S     1      0      0      0      0      0      1 89765 79978 3.2
## 5 Acura RSX      0      0      0      0      0      0      0 23820 21761 2
## 6 Acura TL       0      0      0      0      0      0      0 33195 30299 3.2
## # i 8 more variables: Cylinders <dbl>, Horsepower <dbl>, Weight <dbl>,
## #   Wheelbase <dbl>, Length <dbl>, Width <dbl>, CityMPG <dbl>, HighwayMPG <dbl>
```

The goal is to predict the HighwayMPG from some of the other variables. In order to keep this simple, I will use Horsepower and Weight as predictor variables.

$$\text{HighwayMPG} = \beta_0 + \beta_1 \text{Horsepower} + \beta_2 \text{Weight}$$

We will use the method of least squares to figure out the coefficients (i.e. β_0 , β_1 and β_2).

The data set has 387 observations, I'll use 310 of them for the training data and the remaining 77 observations for the test set. The mean squared error will be used to obtain the prediction error.

```
# Grab 310 observations for the training set
set.seed(12345)
trainvals <- sample(1:387, 310, replace=FALSE)

# Extract the 310 observations from the data set
```

```

train <- cars[trainvals,]

# Put the other 77 values in the test set
test <- cars[-trainvals,] # Note, the "minus sign" in front of trainvals

# Fit a linear model to the training data
fit <- lm(HighwayMPG~Weight+Horsepower,data=train)

# Predict the HighwayMPG at each of the test values.
test$predictions <- predict(fit,test)

# Determine the MSE
MSE <- mean((test$HighwayMPG-test$predictions)^2)
MSE

```

```
## [1] 7.181673
```

Notice that if we ran the above code again, we would get a different MSE because we would have different test/train sets.

Here we run it 5 times, and printout the MSE each time.

```

for (i in 1:5) {
  # Grab 310 observations for the training set
  trainvals <- sample(1:387,310,replace=FALSE)

  # Extract the 310 observations from the data set
  train <- cars[trainvals,]

  # Put the other 77 values in the test set
  test <- cars[-trainvals,] # Note, the "minus sign" in front of trainvals

  # Fit a linear model to the training data
  fit <- lm(HighwayMPG~Weight+Horsepower,data=train)

  # Predict the HighwayMPG at each of the test values.
  test$predictions <- predict(fit,test)

  # Determine the MSE
  MSE <- mean((test$HighwayMPG-test$predictions)^2)
  print(MSE)
}

```

```

## [1] 15.54691
## [1] 8.358736
## [1] 26.5533
## [1] 9.705237
## [1] 12.06945

```

For each test/train split you get a different test error. Typically we run lots and lots of tests and then average the results.

K-fold Cross Validation

In K-fold cross validation, we divide our data into K equal or nearly equal size fold. Suppose using the previous dataset of 387 observations, suppose we want to do 10-fold cross validation. This means that each

fold should have 38.7 data points. Since 10 doesn't go into 387 equally, some folds might have 387 and others 388.

In Python, we typically use scikit-learn to create and manage folds.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold

# Load Data
cars = pd.read_csv("Data/cars.csv")

# Set up 10-fold cross validation
kf = KFold(n_splits=10, shuffle=True, random_state=999)

# Example: look at the first fold
train_index, test_index = next(kf.split(cars))

train = cars.iloc[train_index]
test = cars.iloc[test_index]
train.head()
```

```
##           Car Sports SUV ... Width CityMPG HighwayMPG
## 0      Acura 3.5 RL      0   0 ...   72      18         24
## 1  Acura 3.5 RL Navigation      0   0 ...   72      18         24
## 2           Acura MDX      0   1 ...   77      17         23
## 3      Acura NSX S      1   0 ...   71      17         24
## 5           Acura TL      0   0 ...   72      20         28
##
## [5 rows x 19 columns]
```

Now, we perform 10-fold CV using a simple linear model.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Define response and predictors
X = cars[['Weight', 'Horsepower']]
y = cars['HighwayMPG']

# Initialize storage for results
mse = np.zeros(10)

# Perform cross-validation
for i, (train_index, test_index) in enumerate(kf.split(X)):

    # Split into training and testing sets
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # Train model
    model = LinearRegression()
    _ = model.fit(X_train, y_train)

    # Make predictions
```

```

predictions = model.predict(X_test)

# Compute MSE
mse[i] = mean_squared_error(y_test, predictions)

# Average test error across folds
mse_mean = mse.mean()
print(f"{mse_mean:.5f}")

## 11.08847

```

Leave One Out Cross Validation (LOOCV)

Typical cross-validation splits the data into a test/train set. A typical split is usually 80/20. As we saw, the MSE is highly dependent on the random values selected for the training set. LOOCV works as follows.

1. Leave out point 1 (i.e. $(\mathbf{x}_1, y_1)$).
2. Fit the model on the remaining $n - 1$ points (i.e. $(\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3), \dots, (\mathbf{x}_n, y_n)$).
3. Use the model you fit in step 2 to predict y_1 using $\mathbf{x}_1$ as the input vector. Call the prediction $\hat{y}_1$.
4. Calculate the MSE for point 1 (i.e. $\text{MSE}_1 = (y_1 - \hat{y}_1)^2$).
5. Repeat the process for the remaining $n - 1$ points.

The LOOCV estimate for the test MSE is calculated by:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i$$

One can easily see that LOOCV is computationally expensive. It requires fitting the model n times. Once for each point in the data set. K-fold cross-validation is less computationally intensive.

Bias Variance Trade-off for K-fold Cross-Validation

One advantage of leave-one-out cross-validation (LOOCV) is that each model is trained on $n - 1$ observations. As a result, LOOCV provides an estimate of test error with very low bias, since the fitted models closely resemble the model trained on the full dataset. From a bias standpoint, larger values of K and LOOCV in particular—are therefore preferred.

From a variance standpoint, however, LOOCV can exhibit higher variance than K -fold cross-validation. In LOOCV, each training set differs from the others by only a single observation. For unstable learning algorithms, leaving out one influential or high-leverage point can substantially change the fitted model, leading to large fluctuations in the estimated test error across datasets. Some general observations regarding the bias-variance trade-off are:

Bias typically decreases as K increases, though this is not guaranteed in every setting. Many texts recommend $K = 5$ or $K = 10$ as a practical compromise that avoids both high bias and high variance.

As the number of folds increases, each training set becomes larger and more representative of the full dataset, which generally leads to lower bias in the cross-validation estimate.

In many real-world problems involving unstable learners, variance often increases as K increases, particularly for large K or LOOCV.

For unstable learners (e.g., decision trees), small changes in the training data can lead to large changes in the fitted model. As K increases in cross-validation (reducing the size of each validation set), this instability is reflected more strongly in the estimated error, resulting in higher variance.

As the number of folds K increases, the bias of cross-validation typically decreases. The effect on variance, however, depends on model stability and the source of randomness, so it may increase, decrease, or remain roughly unchanged.

Overfitting

A model is **overfitting** the data when it captures the true signal, but also captures the random noise in the training data. The result is that the training error can be very low, but the test error ends being quite high.

You can see from this example, the degree 12 model has a fairly small training error. However, the test error is huge!! Way overfit.

Training vs. Test Error

Let $\hat{f}(x)$ be the fitted model. Typically, we fit the model so as to minimize the difference between the actual response value (y_i) and the fitted value $\hat{f}(x_i)$. This optimistically biases our estimate.

Recall from the lecture on estimator properties, it's possible to find the “sweet” spot. That said, we can minimize the MSE of the test set, but there is a tradeoff between bias and variance.

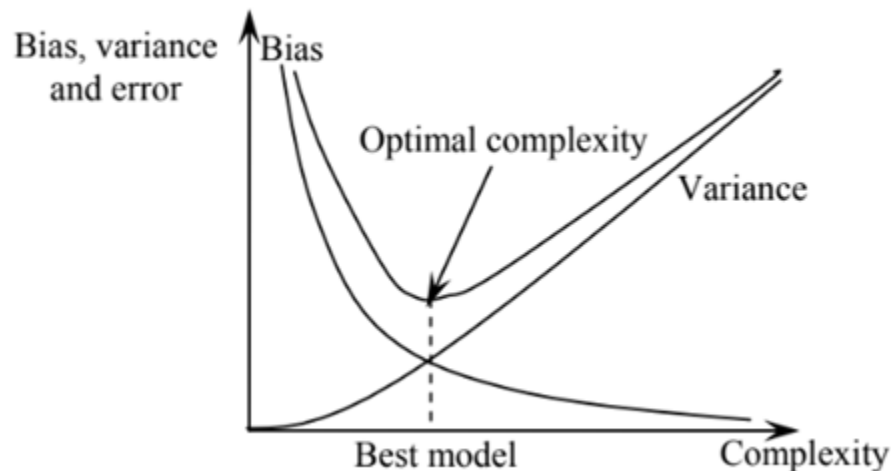


Figure 1: Bias Variance Tradeoff

From this plot, what you see is training error decreases monotonically, CV error decreases then increases. The minimum CV errors gives the best model.

- **Model Flexibility:** A model's capacity to bend and adapt to the data's shape, capturing nonlinearities, interactions, and nuances. Think of a wiggly line vs. a straight line (less flexible).
- **Model Complexity:** The technical “size” or “degrees of freedom” of a model, often measured by the number of parameters or the richness of its functional form (e.g., high-degree polynomials, deep neural networks, many splines).

Because of the bias variance tradeoff, the following is generally true:

→ Overfitting is fundamentally about variance:

Simple models:



Figure 2: Interpretability vs. Flexibility

- High bias, low variance
- Underfit

Complex models: + Low bias, high variance Overfit

When a model is overfit, the variance dominates!

Why Overfitting Happens

There are 3 key reasons why overfitting happens:

1. Too much flexibility
 - High-degree polynomials
 - Deep trees
 - Unregularized models
2. Limited data
 - Not enough information to estimate complex patterns
3. Noise in the data
 - The model mistakes noise for signal

A model that fits the data best **is not** necessarily the model that predicts best.

Enter Cross-Validation

This is exactly why we introduced cross-validation in the previous part of this lecture.

Training error:

- Too optimistic
- Cannot detect overfitting

Cross-validation:

- Mimics out-of-sample performance
- Estimates test error

Summary: Overfitting occurs when a model learns noise in the training data, leading to low training error but poor predictive performance. As model complexity increases, training error always decreases, but test error eventually increases due to high variance. Cross-validation provides an estimate of test error and helps identify the optimal level of model complexity.